C++ MASTER REFERENCE

SYNTHÈSE ARCHITECTURE, MÉMOIRE ET ORIENTÉ OBJET | NIVEAU INGÉNIEUR

1. La Mémoire et la Portée (Scope)

Le C++ ne possède pas de "Garbage Collector" automatique global comme Java ou Python. Le développeur est le seul responsable de la création et de la destruction de la mémoire. La mécanique fondamentale qui régit la vie d'une variable ou d'un objet est la **Portée (Scope)**, définie par les accolades `{ }`.

```
int main() { // La zone mémoire Stack globale commence { // Ouverture d'un sous-scope SmartCard ma_carte(1234); // Naissance de l'objet } // <-- CRITIQUE : A l'accolade fermante, l'objet est détruit. ma_carte.deverouiller(); // ERREUR FATALE : L'objet n'existe plus. return 0; }
```

🚨 **Règle d'or de l'Architecte :** Tout ce qui naît à l'intérieur d'une paire d'accolades meurt irrémédiablement lorsque l'ordinateur franchit l'accolade fermante. C'est la mémoire *Stack* (Pile).

2. Les Références (&) et la Sécurité

Le passage par référence permet de créer un *Alias* (un tunnel direct) vers une variable existante en mémoire, plutôt que d'en faire une photocopie coûteuse. Cela est vital pour les performances, mais extrêmement dangereux si mal utilisé, notamment avec l'Encapsulation.

Faille critique : Ne renvoyez jamais une variable privée via un getter utilisant une référence (`int& getCle()`). Cela permettrait au programme principal de modifier la variable privée directement, détruisant ainsi l'intégrité du coffre-fort de la classe.

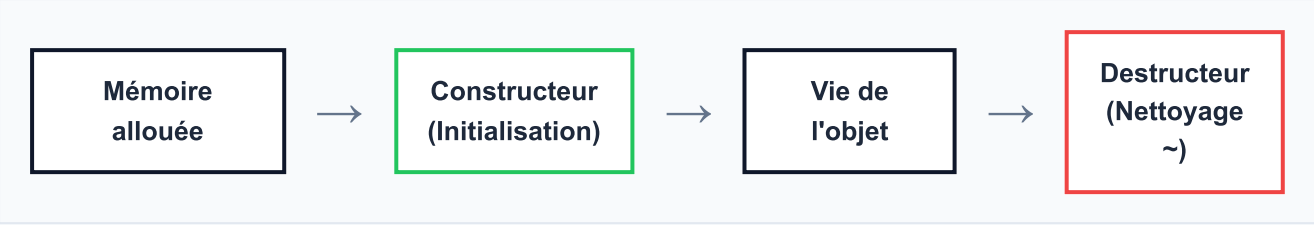
3. L'Encapsulation et les Droits d'Accès

L'encapsulation est le principe de cacher les rouages internes d'un objet. Le C++ propose trois niveaux de confidentialité stricts :

Mot-clé	Accès depuis la Classe	Accès depuis les Filles (Héritage)	Accès depuis le main()
`private`	Oui (Secret absolu)	**Non** (Bloqué)	Non
`protected`	Oui	**Oui** (L'ADN familial)	Non
`public`	Oui	Oui	**Oui** (Interface libre)

4. Le Cycle de Vie : Constructeurs & Destructeurs

Un objet ne peut exister sans être initialisé, et ne peut disparaître sans nettoyer sa mémoire. Le Constructeur et le Destructeur sont appelés automatiquement par le système.



> **Le Cadeau Retiré du Compilateur :**
> Par défaut, le compilateur offre un constructeur vide (ex: `Serveur()`). Mais *dès la seconde* où vous codez un constructeur personnalisé (ex: `Serveur(string ip)`), le compilateur retire son cadeau. Il devient alors impossible de créer l'objet sans fournir l'IP.

Le Destructeur (`~Classe()`) : Il est l'ultime fonction appelée avant la mort de l'objet. Il ne prend jamais de paramètres et ne renvoie rien. Il sert à fermer les connexions réseaux, sauvegarder des logs de sécurité, ou libérer la mémoire manuelle (Heap).

5. L'Héritage et l'Ordre de Naissance

L'héritage résout la duplication de code (principe DRY). Une classe Fille hérite des propriétés et méthodes de sa Mère. Cependant, la loi physique de la mémoire exige que **la Mère soit toujours construite avant la Fille**.

Les 4 Cas d'Initialisation (Syntaxe des deux-points `:`)

La liste d'initialisation intercepte la construction avant même l'entrée dans les accolades de la classe Fille.

Cas Métier	Mécanisme et Code
1. L'Appel Fantôme (La Mère a un constructeur vide)	La Fille ne fait rien de spécial. Le compilateur appelle silencieusement le constructeur de la Mère. `Fille() { ... }`
2. Le Relais Forcé (Valeur hardcodée)	La Fille connaît la donnée par défaut et l'envoie en dur à sa mère. `Moto() : Vehicule(2) { ... }`
3. Le Facteur (Pass-through)	La Fille demande une donnée à l'utilisateur uniquement pour la transmettre à la Mère. `Medecin(string n) : Employe(n) { ... }`
4. Le Triage (L'Architecte)	La Fille reçoit tout, envoie une partie à la Mère (via `:`), et garde le reste pour ses accolades `{ }`. `Camion(string m, int p) : Vehicule(m) { poids = p; }`

> **Attention à la Chronologie :** Ne tentez *jamais* de construire la Mère à l'intérieur des accolades `{ }` de la Fille. Les accolades servent à aménager l'objet, mais les fondations (la Mère) doivent déjà être posées via les deux-points `:`.

6. Masquage et Polymorphisme (La Télécommande)

Une classe Fille peut redéfinir l'action de sa Mère. Pour ajouter au comportement plutôt que de tout écraser, on utilise la résolution de portée : `Mere::action();`. Pour que le système reconnaisse dynamiquement la vraie nature de l'objet derrière un pointeur générique, on utilise `virtual` et `override`.

```
class Vehicule { public: // virtual permet au compilateur de vérifier le vrai type en mémoire virtual void klaxonner() { cout << "Tut tut"; } }; class Camion : public Vehicule { public: // override confirme de manière stricte le remplacement void klaxonner() override { cout << "POUET !"; } }; int main() { Vehicule* flotte = new Camion(); flotte->klaxonner(); // Affiche "POUET !" grâce au virtual. delete flotte; // Libération de la mémoire Heap }
```

7. Hygiène Industrielle et Outils

Le code d'un ingénieur doit être traçable et strict. Les fichiers compilés (binaires) ne doivent jamais être envoyés sur le réseau.

```
# Compilation ultra-stricte pour traquer les failles : g++ -Wall -Wextra -pedantic main.cpp -o programme # Fichier .gitignore minimal pour C++ : *.o *.out *.exe
```

Ne poussez sur Git (`git add`, `git commit`, `git push`) que les fichiers sources (`.cpp`, `.hpp`). La machine de l'utilisateur final se chargera de la compilation.